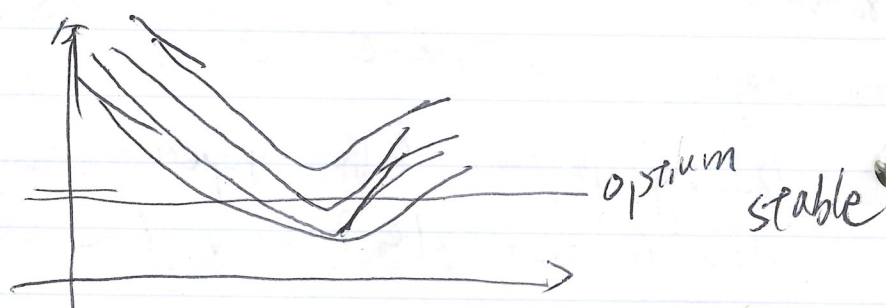
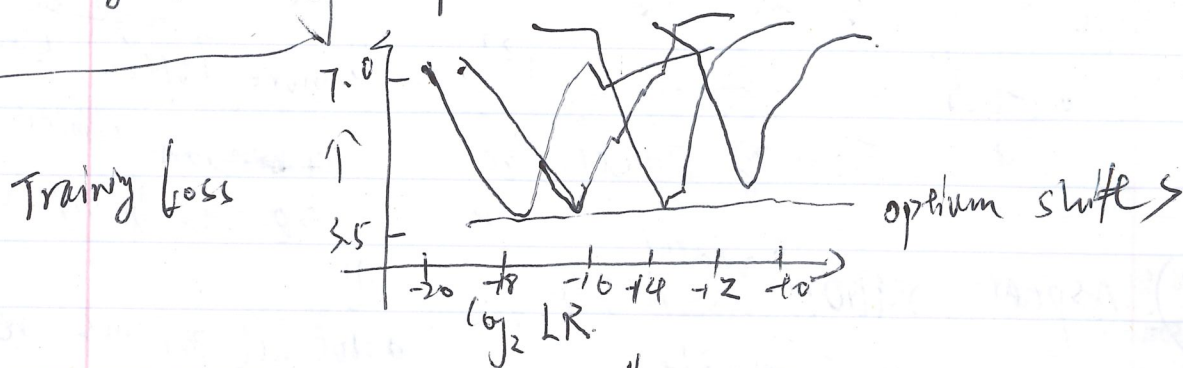


As we increase both compute and model size,
 how should we scale training!
 { Huge Batches, same number of steps
 { Fixed batches, more steps

(5), Learning Rates: map and scale-aware LR choices



if we naively scale up $\rightarrow$ optimal learning rate depends on scale.
 we need scaling aware initialization and learning rate scaling!

Caution: scaling behaviors can differ downstream

{ So far: scaling is predictable and depends mainly on parameters
 { catch: downstream scaling can often be much less predictable

Summary: { Data scaling: understand how data affects models (lean theory)
 { Model scaling: dramatically reduce costs for training
 { scaling as prediction: what problems can be "brute forced".

~~super~~ supervising calls analysis.

- The effect of hyperparameters on big LMs can be predicted before training

=> Optimizer choice

=> Model depth

=> Architecture choice

The scaling law based design procedure:

1: Train a few smaller models

2: Establish a scaling law (e.g. ADAM vs SGD scaling law)

3: Select optimal hyperparam based on scaling law prediction

- Do we need more data or larger model?

more data $\Leftrightarrow$ ~~no~~ larger model $\Leftrightarrow$ $\left\{ \begin{array}{l} \text{compute} \\ \text{loss convergence} \\ \text{speed} \end{array} \right.$

less data $\Leftrightarrow$ smaller model

Compute trade-off: [compute versus performance]

"optimal" scaling law is hard to get!

diffusion models: IsoFlops are pretty easy to execute
and our group has replicated these results

Summary: scaling laws for models and compute

=> log-linearity extends to model parameters and compute

small models: pick optimizer/architecture and sizes / $\left\{ \begin{array}{l} \text{Big model vs data} \\ \text{resources trade-off} \end{array} \right.$